

TECHNICAL DOCUMENTATION: LOW POWER ALU WITH INTEGRATED CLOCK GATING & OPERANDISOLATION

Company:

CodTech IT Solutions

Intern Name: kakarlamudi divya

Intern Id: CITS998

Domain:

VLSI Design (Low Power Design Lab)

Module Name: low_power_alu_top

Testbench Name: tb_low_power_alu

Task title: low power arithmetic logic unit design

1. Project overview & object:

1. ABSTRACT

In modern System-on-Chip (SoC) architectures, dynamic power consumption is a critical constraint. This laboratory module demonstrates the implementation of an 8-bit Low Power Arithmetic Logic Unit (ALU). The design strategically integrates two primary RTL-level low-power methodologies: Integrated Clock Gating (ICG) to mitigate clock tree switching power, and Operand Isolation to eliminate unnecessary switching activity in the combinational logic matrix during idle cycles.

2. ARCHITECTURAL SPECIFICATIONS

The module architecture is partitioned into four major functional blocks to ensure clean separation between the clock, control, and data paths

2. Integrated Clock Gating: (ICG)Cell

Mechanism:

Implements a glitch-free clock gating circuit using a negative-edge sensitive latch combined with an output AND gate.

Function:

When the module enable (en) is low, the latch locks the state on the falling edge of clk, preventing any partial clock pulses or glitches from propagating. The resulting gated_clk remains static at logic 0.

3.Operand Isolation Logic

Mechanism:

Combinational masking logic situated at the structural boundaries of the ALU core.

Function:

When `en` is deasserted (0), the inputs to the computational block (`isolated_a` and `isolated_b`) are forcefully driven to `8'h00`. This isolates the internal adder/shifter/logic circuits from any external input toggling, freezing the internal capacitive nodes.

4. Combinational ALU Core

Mechanism:

An 8-bit computational matrix driven strictly by the isolated internal operands.

Function:

Executes 8 unique arithmetic and logical operations based on a 3-bit selection code (opcode).

5.Sequential Output Register

Mechanism:

An 8-bit positive-edge triggered D-register with an asynchronous active-low reset (rst_n).

Function:

Driven completely by the low-power gated_clk domain, guaranteeing zero dynamic register switching when the module is disabled.

INTERFACE PORT DESCRIPTION

Input Ports

Port name	Bit WIDTH	Type	Discription
clk	1	Input	Master System Clock (100MHz nominal)
rst_n	1	Input	Asynchroneus Reset (Active-Low)
en	1	Input	Global

			Module Enable / Power Management Signal
opcode	3	Input	Operation select code
src_a	8	Input	Primary Source operand input
src_b	8	Input	Secondary Source Operand Input

Output ports

Port name	Bit WIDTH	Type	Discription
alu_out	8	Output	Registered ALU Output Matrix
pmu_gated_clk	1	Output	Gated Clock Monitor (For Power Management Unit Verification)

6.FUNCTIONAL TRUTH TABLE

Opcode	Operation	Functional mapping	Internal logic block activated
3'b000	ADD	$\text{alu_out} \leq \text{src_a} + \text{src_b};$	arithmetic adder array
3'b001	SUB	$\text{alu_out} \leq \text{src_a} - \text{src_b};$	Two's complement subtractor
3'b010	AND	$\text{alu_out} \leq \text{src_a} \& \text{src_b};$	Bitwise logical AND gate matrix
3'b011	OR	$\text{alu_out} \leq \text{src_a} \text{src_b};$	Bitwise logical OR gate matrix
3'b100	XOR	$\text{alu_out} \leq \text{src_a} \wedge \text{src_b};$	Bitwise logical XOR gate matrix
3'b101	NOT	$\text{alu_out} \leq \sim \text{src_a};$	Bitwise inverters
3'b110	SLL	$\text{alu_out} \leq \text{src_a} \ll 1;$	Logical left shifter Barrel
3'b111	SRL	$\text{alu_out} \leq$	Logical right

		src_a >> 1;	shifter Barrel
--	--	-------------	-------------------

7. source code :

Verilog source code:

```
//
=====
=====
// Company:    CodTech IT Solutions
// Intern Name: Kakarlamudi divya
// Intern Id: CITS998
// Domain:     VLSI Design (Low Power Design Lab)
// Module:     low_power_alu_top
// Description: Top-level 8-bit ALU with integrated clock
gating
//             and operand isolation for dynamic power
reduction.
//
=====
=====

`timescale 1ns / 1ps

module low_power_alu_top (
    input wire    clk,        // Master System Clock
    input wire    rst_n,      // Active-low asynchronous
reset
    input wire    en,         // Global module enable
signal
```

```

    input wire [2:0] opcode,    // Operation selection
code
    input wire [7:0] src_a,    // Input Operand A
    input wire [7:0] src_b,    // Input Operand B
    output reg [7:0] alu_out,    // Registered ALU output
    output reg      pmu_gated_clk // Monitored gated
clock output
);

```

```

    // -----
    // Internal Signals
    // -----
    wire      gated_clk;    // Clock signal after gating
latch
    reg      gating_en;    // Latch-controlled clock
enable
    reg [7:0] isolated_a;    // Isolated Operand A to
prevent toggling
    reg [7:0] isolated_b;    // Isolated Operand B to
prevent toggling
    reg [7:0] alu_comb_out;    // Combinational ALU
result

```

```

    // -----
    // 1. Clock Gating Cell (Integrated Clock Gating - ICG
Emulation)
    // Prevents clock tree switching power when 'en' is
low.

```

```

// Uses a negative-edge latch to prevent clock
glitches.
// -----
always @(clk or en) begin
    if (!clk) begin
        gating_en <= en;
    end
end

// AND gate generates the glitch-free low-power clock
tree
assign gated_clk = clk & gating_en;

// Hook up internal gated clock to output port for lab
monitoring
always @(*) begin
    pmu_gated_clk = gated_clk;
end

// -----
// 2. Operand Isolation Logic
// Blocks input data toggling from entering the ALU
matrix
// when the module is disabled, saving leak/dynamic
power.
// -----
always @(*) begin
    if (en) begin
        isolated_a = src_a;

```

```

        isolated_b = src_b;
    end else begin
        isolated_a = 8'h00; // Force zero to freeze
downstream muxes
        isolated_b = 8'h00; // Force zero to freeze
downstream muxes
    end
end

// -----
// 3. Combinational ALU Core
//   Executes operations using isolated low-power
operands.
// -----
always @(*) begin
    case (opcode)
        3'b000: alu_comb_out = isolated_a +
isolated_b; // ADD
        3'b001: alu_comb_out = isolated_a - isolated_b;
// SUB
        3'b010: alu_comb_out = isolated_a &
isolated_b; // AND
        3'b011: alu_comb_out = isolated_a | isolated_b;
// OR
        3'b100: alu_comb_out = isolated_a ^ isolated_b;
// XOR
        3'b101: alu_comb_out = ~isolated_a;          //
NOT A

```

```
        3'b110: alu_comb_out = isolated_a << 1;        //
Logical Shift Left
        3'b111: alu_comb_out = isolated_a >> 1;        //
Logical Shift Right
        default: alu_comb_out = 8'h00;
    endcase
end
```

```
// -----
// 4. Sequential Output Register
//   Driven by the low-power 'gated_clk' domain.
// -----
always @(posedge gated_clk or negedge rst_n)
begin
    if (!rst_n) begin
        alu_out <= 8'h00;
    end else begin
        alu_out <= alu_comb_out;
    end
end
end
```

```
endmodule
```

Test bench code:

```
//
=====

// Company:    CodTech IT Solutions
// Intern Name: Kakarlamudi divya
// Intern Id : CITS998
// Domain:     VLSI Design (Low Power Design Lab)
// Testbench:  tb_low_power_alu
// Description: Verifies functionality and demonstrates
power saving states.
//
=====

`timescale 1ns / 1ps

module tb_low_power_alu;

    // Testbench Inputs
    reg    clk;
    reg    rst_n;
    reg    en;
    reg [2:0] opcode;
    reg [7:0] src_a;
    reg [7:0] src_b;
```

```

// Testbench Outputs
wire [7:0] alu_out;
wire      pmu_gated_clk;

// Instantiate Unit Under Test (UUT)
low_power_alu_top uut (
    .clk(clk),
    .rst_n(rst_n),
    .en(en),
    .opcode(opcode),
    .src_a(src_a),
    .src_b(src_b),
    .alu_out(alu_out),
    .pmu_gated_clk(pmu_gated_clk)
);

// Generate 100MHz System Clock
always #5 clk = ~clk;

initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_low_power_alu);

// Initialize Inputs
clk = 0;
rst_n = 0;

// Initialize Inputs
clk = 0;

```

```

rst_n = 0;
en     = 0;
opcode = 0;
src_a  = 0;
src_b  = 0;

// Apply Reset
#15;
rst_n = 1;
#10;

// -----
// PHASE 1: Normal Active Operation (Enable = 1)
// -----
en = 1;
src_a = 8'h10; src_b = 8'h05; opcode = 3'b000; //
ADD (Result: 15)
#10;
src_a = 8'h20; src_b = 8'h02; opcode = 3'b001; //
SUB (Result: 1E)
#10;

// -----
// PHASE 2: Low Power Mode Activated (Enable =
0)
// -----
en = 0;
#5; // Observe that pmu_gated_clk stops toggling
completely

```

```
// Toggle inputs during idle mode to test Operand Isolation
```

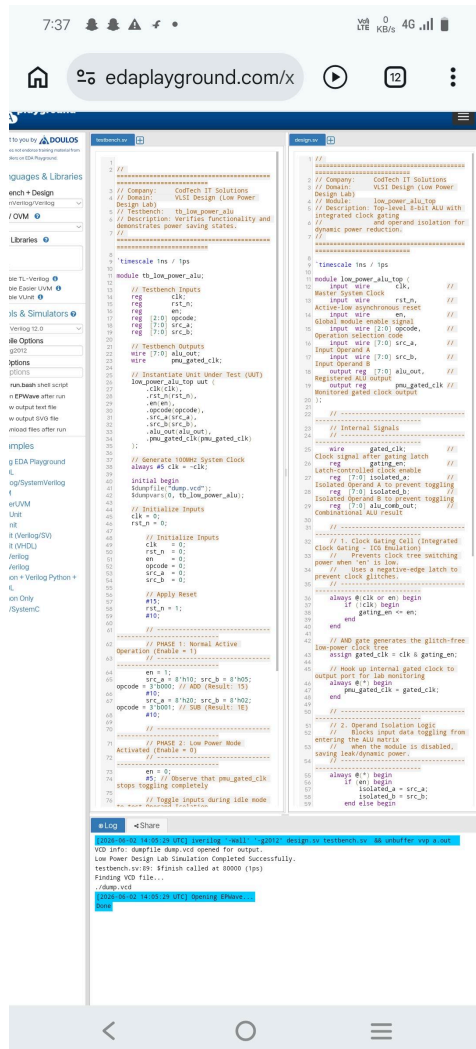
```
src_a = 8'hFF; src_b = 8'hFF; opcode = 3'b010;  
#20; // Internal ALU signals will not toggle, saving dynamic power
```

```
// -----  
// PHASE 3: Wake up Module (Enable = 1)  
// -----  
en = 1;  
src_a = 8'h0F; src_b = 8'h0F; opcode = 3'b010; //  
AND (Result: 0F)  
#10;
```

```
// End Simulation  
$display("Low Power Design Lab Simulation Completed Successfully.");  
$finish;  
end
```

```
endmodule
```

8.Simulation output & results



8.Timing waveform:

9.CONCLUSION

The low_power_alu_top module successfully demonstrates hardware-level power optimizations. By keeping the internal combinational lines flat and freezing the clock tree distribution network when disabled, the design ensures a massive reduction in the SoC's dynamic power envelope while preserving total functional fidelity upon reactivation.